Centro de Investigación y de Estudios Avanzados

Programación Avanzada

Examen

Profesor: Dr. Cuauhtemoc Mancillas López

Estudiante: José Fabián Grajales Ocampo

Fecha: 6 de noviembre de 2026

Problema 1:

El juego Yahtzee utiliza cinco dados, que se lanzan en 13 rondas. La tabla que registra el marcador del juego contiene 13 categorías. Cada ronda, el jugador selecciona en qué categoría se clasifica su puntuación; cada categoría debe ser seleccionada sólo una vez en el juego y antes de tirar los dados. Las categorías son:

- **Unos**. Suma de todos los unos obtenidos.
- **Dos**. Suma de todos los dos obtenidos.
- **Tres**. Suma de todos los tres obtenidos.
- **Cuatros**. Suma de todos los cuatros obtenidos.
- **Cincos**. Suma de todos los cincos obtenidos.
- **Seises**. Suma de todos los seises obtenidos.
- **Suerte**. Suma de todos dados.
- **Tres de un tipo**. Suma de todos los dados en los que al menos tres tengan el mismo valor.
- **Cuatro de un tipo.** Suma de todos los dados en los que al menos cuatro tengan el mismo valor.
- **Cinco de un tipo.** 50 puntos, si al menos cinco dados tienen el mismo valor.
- **Secuencia corta.** 25 puntos, si los dados forman alguna de las secuencias 1,2,3,4 o 2,3,4,5 o 3,4,5,6.
- **Secuencia larga.** 35 puntos, si los dados forman alguna de las secuencias 1,2,3,4,5 o 2,3,4,5,6.
- **Full house.** 40 puntos, tres dados iguales y dos dados iguales, por ejemplo: 5,5,5,2,2 o 6,6,6,1,1.

En las últimas seis categorías se obtienen cero puntos si el criterio no se cumple. El puntaje final es la suma de las trece categorías, más un bono de 35 puntos si la suma de las primeras 6 categorías es de 63 o más. Simular el juego generando los dados de forma aleatoria, guardar en un archivo de texto los valores de los dados generados para 1 las 13 rondas, la puntuación de cada una de las categorías, las puntuación total y el bono de puntuación referente a las primeras categorías. Correr muchas veces el programa, por ejemplo 1000 y obtener las puntuaciones máxima, mínima y más común.

Lenguaje: versión en C++ y versión en Python.

Solución:

El problema dice que cada categoría debe ser seleccionada sólo una vez en el juego y debe ser antes de tirar los dados por lo que podemos jugar en cada una de las categoría, en la ronda uno en la categoría uno y así sucesivamente hasta la ronda 13.

Entonces necesitamos iterar 13 veces porque son 13 rondas totales. Los dados tienen números entre 1 y 6 y se lanzan 5 veces por lo que debemos de generar 5 números aleatorios entre 1 y 6. De alguna manera debemos evaluar los puntajes obtenidos de esos 5 lanzamientos según la categoría o ronda en la que estemos. Guardar los datos y el puntaje obtenido. En las indicaciones dice que el puntaje final es la suma del resultado de las 13 rondas más un bono de 35 puntos si la suma de las primeras 6 categorías es de 63 o más, por lo tanto si la suma de las categorías 1 a 6 es $\geq$ 63 sumar 35 puntos extra. Por último, calcular el puntaje total y guardarlo en un archivo de texto.

En probabilidad, estamos realizando simulaciones aplicando la simulación Monte Carlo que básicamente consiste en repetir un número de veces un experimento y en este caso se piden 1000 partidas y se puede aplicar el mismo concepto, que es básicamente realizar 1000 iteraciones. Para eso podemos almacenar el puntaje final de las 1000 partidas en una lista o arreglo, y cuando tengamos eso calcular el valor máximo, valor mínimo y el más común (la moda).

A manera de **pseudocódigo** el programa realiza lo siguiente:

```
FUNCION evaluar_categoria(dados, categoria):
    frecuencias = contar_frecuencias(dados) # Retorna cuántas veces sale cada
número del 1 al 6
    suma_total = sumar_todos(dados)

    según sea la categoria:
        CASO 1 a 6 (Unos a Seises):
            RETORNAR frecuencias[categoria] * categoria
        CASO 7 (Suerte):
            RETORNAR suma_total
        CASO 8 (Tres de un tipo):
            SI algun valor en frecuencias es >= 3:
                RETORNAR suma_total
            SINO:
                RETORNAR 0
        CASO 9 (Cuatro de un tipo):
            SI algun valor en frecuencias es >= 4:
                RETORNAR suma_total
            SINO:
                RETORNAR 0

        CASO 10 (Cinco de un tipo):
            SI algun valor en frecuencias es == 5:
                RETORNAR 50
            SINO:
                RETORNAR 0
        CASO 11 (Secuencia corta):
```

```
            dados_unicos = ordenar_y_eliminar_duplicados(dados)
            SI dados_unicos contiene (1,2,3,4) O (2,3,4,5) O (3,4,5,6):
                RETORNAR 25
            SINO:
                RETORNAR 0
        CASO 12 (Secuencia larga):
            dados_unicos = ordenar_y_eliminar_duplicados(dados)
            SI dados_unicos contiene (1,2,3,4,5) O (2,3,4,5,6):
                RETORNAR 35
            SINO:
                RETORNAR 0
        CASO 13 (Full house):
            SI (frecuencias contiene un 3 Y frecuencias contiene un 2):
                RETORNAR 40
            SINO:
                RETORNAR 0
FUNCION jugar_partida(archivo_salida):
    puntajes_ronda = un arreglo vacio de tamaño 13
    suma_primeras_6 = 0

    PARA ronda DESDE 1 HASTA 13:
        dados = generar_5_numeros_aleatorios(1 a 6)
        puntaje = evaluar_categoria(dados, ronda)
        puntajes_ronda[ronda - 1] = puntaje

        SI ronda <= 6:
            suma_primeras_6 = suma_primeras_6 + puntaje

        Escribir en archivo_salida: "Ronda X - Dados: [dados] - Categoria X -
Puntaje: Y"

    bono = 0
    SI suma_primeras_6 >= 63:
        bono = 35

    puntaje_total = sumar(puntajes_ronda) + bono

    Escribir en archivo_salida: "Bono: [bono] | Puntaje Total:
[puntaje_total]"
    Escribir en archivo_salida:
"-----------------------------------------"
```

```
    RETORNAR puntaje_total

FUNCIÓN PRINCIPAL:
    Abrir archivo_texto para escritura
    N_simulaciones = 1000
    historial_puntajes = arreglo vacio

    PARA i DESDE 1 HASTA N_simulaciones:
        puntaje_final = jugar_partida(archivo_texto)
        Agregar puntaje_final a historial_puntajes

    Cerrar archivo_texto
  //aquí calculamos el máximo, mínimo y moda mediante el arreglo //
historial_punutajes
    maximo = calcular_maximo(historial_puntajes)
    minimo = calcular_minimo(historial_puntajes)
    moda = calcular_mas_comun(historial_puntajes)
    //imprimimos en consola
    Imprimir "Simulación terminada."
    Imprimir "Puntuación máxima: ", maximo
    Imprimir "Puntuación mínima: ", minimo
    Imprimir "Puntuación más común: ", moda
```

Compilación y ejecución del Código:

El archivo fue nombrado como P1_FabianGrajales.cpp

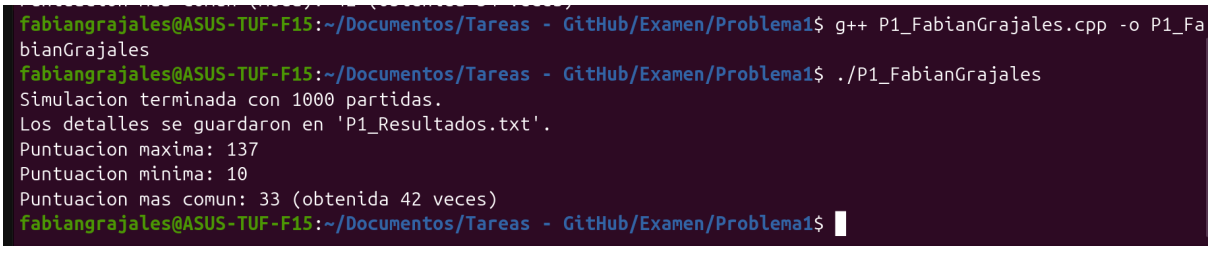
- g++ P1_FabianGrajales.cpp -o P1_FabianGrajales
- ./P1_FabianGrajales

Para el archivo python fue nombrado como

- python3 P1_FabianGrajales_Python.py

Resultados:

a) Resultados obtenidos en C++



b) Resultados obtenidos en python:

```
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema1$ ^C
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema1$ python3 P1_FabianGrajales_Python.p
y
Simulacion terminada con 1000 partidas.
Los detalles se guardaron en 'P1_Resultados_Python.txt'.
Puntuacion maxima: 153
Puntuacion minima: 12
Puntuacion mas comun (Moda): 28 (obtenida 33 veces)
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema1$
```

Problema 2:

El método de cifrado más sencillo consiste en permutar el alfabeto y sustituir cada carácter original por el correspondiente del alfabeto permutado. El alfabeto español tiene 27 letras, por lo que hay 27! posibles permutaciones.

Generar un programa que reciba como entrada un texto en español (al menos tres páginas de tamaño) mediante un archivo de texto y obtenga el mensaje cifrado mediante la permutación del alfabeto, dicha permutación será generada de manera aleatoria y almacenada en un archivo para poder recuperar el mensaje original. Igualmente, hacer un programa que, a partir del mensaje cifrado y la permutación almacenada, recupere el mensaje original.

Lenguaje: C.

Genere un programa que reciba ambos archivos, el mensaje original y el cifrado, y calcule el histograma de las letras que aparezcan en ambos archivos; vea si es posible, a partir de la comparación de ambos histogramas, descifrar el mensaje. Graficar ambos histogramas para el reporte. **Lenguaje: Python utilizando un contenedor (ej: tabla hash).**

Solución:

Obviamente el alfabeto español contiene la letra ñ, siendo el abecedario completo:

abcdefghijklmnñopqrstuvwxyz

Para realizar este problema lo haŕe definiendo el abecedario como un arreglo estático que no incluye mayúsculas ni acentos. Necesitamos una permutación aleatoria de este alfabeto, una función que tome el alfabeto original, lo desordene y guarde ese alfabeto en un archivo diferente que será la clave.

Para el proceso de cifrado hay que leer el archivo caracter por caracter y por ejemplo, si la letra es una c, en la posición del arreglo 2, escribimos en el archivo de salida la letra de esa posición pero en el alfabeto permutado. Los espacios, números y signos se dejan intactos.

Para el proceso de descifrado, es el proceso al revés, leemos el archivo cifrado, si el carácter es una letra, buscamos su posición en el alfabeto permutado, es decir, la clave guardada y escribimos en la salida la letra correspondiente al alfabeto original.

Para la segunda parte, se pide usar una tabla hash en python y graficar con un histograma las frecuencias de aparición de las letras.

Parte 1: C (Cifrado y Descifrado)

```
CONSTANTE ALFABETO_ORIGINAL = "abcdefghijklmnñopqrstuvwxyz"

FUNCION Generar Permutación():
    alfabeto_permutado = copiar(ALFABETO_ORIGINAL)
    Para i desde 0 hasta 26:
        j = numero_aleatorio(0, 26)
        intercambiar(alfabeto_permutado[i], alfabeto_permutado[j])

    Abrir archivo "clave.txt" para escritura
    Escribir alfabeto_permutado en "clave.txt"
    Cerrar archivo
    RETORNAR alfabeto_permutado

FUNCION Cifrar(archivo_entrada, archivo_salida, clave_permutada):
    Abrir archivo_entrada en modo lectura
    Abrir archivo_salida en modo escritura

    MIENTRAS no sea el fin de archivo_entrada:
        caracter = Leer_siguiente_caracter(archivo_entrada)

        SI caracter está en ALFABETO_ORIGINAL:
            indice = buscar_posicion(caracter, ALFABETO_ORIGINAL)
            caracter_cifrado = clave_permutada[indice]
            Escribir caracter_cifrado en archivo_salida
        SINO:
            // Es un espacio, punto, coma, salto de linea, etc.
            Escribir caracter en archivo_salida

    Cerrar ambos archivos

FUNCION Descifrar(archivo_cifrado, archivo_salida, clave_permutada):
    Abrir archivo_cifrado en modo lectura
    Abrir archivo_salida en modo escritura

    MIENTRAS no sea el fin de archivo_cifrado:
        caracter = Leer_siguiente_caracter(archivo_cifrado)

        SI caracter está en clave_permutada:
            indice = buscar_posicion(caracter, clave_permutada)
```

```
            caracter_original = ALFABETO_ORIGINAL[indice]
            Escribir caracter_original en archivo_salida
        SINO:
            Escribir caracter en archivo_salida

    Cerrar ambos archivos

ALGORITMO PRINCIPAL EN C:
    clave = Generar_Permutacion()
    Cifrar("texto_original.txt", "texto_cifrado.txt", clave)

    // Simular que leemos la clave del archivo para descifrar
    clave_leida = Leer_desde_archivo("clave.txt")
    Descifrar("texto_cifrado.txt", "texto_descifrado.txt", clave_leida)
```

Parte 2: Python (Histogramas y Gráficas)

```
IMPORTAR matplotlib.pyplot como plt
IMPORTAR colecciones.Counter como TablaHash

FUNCION Calcular_Frecuencias(ruta_archivo):
    Abrir ruta_archivo como archivo en modo lectura
    texto = Leer_todo(archivo)
    Convertir texto a minúsculas

    Filtro: dejar solo las letras del alfabeto (quitar espacios, números,
puntos)
    texto_filtrado = extraer_solo_letras(texto)

    // Counter implementa la tabla hash que pide el ejercicio
    histograma = TablaHash(texto_filtrado)
    RETORNAR histograma

ALGORITMO PRINCIPAL EN PYTHON:
    // Obtener los diccionarios (tablas hash) de frecuencias
    frecuencias_original = Calcular_Frecuencias("texto_original.txt")
    frecuencias_cifrado = Calcular_Frecuencias("texto_cifrado.txt")

    // Ordenar las frecuencias de mayor a menor para facilitar la comparación
    ordenado_original = ordenar_valores_descendente(frecuencias_original)
    ordenado_cifrado = ordenar_valores_descendente(frecuencias_cifrado)
```

```
    // Imprimir conclusión sobre el descifrado visual
    Imprimir "Comparación de las 5 letras más frecuentes:"
    Imprimir "Original: ", los_5_primeros(ordenado_original)
    Imprimir "Cifrado: ", los_5_primeros(ordenado_cifrado)
    Imprimir "¿Es posible descifrarlo? Sí, emparejando los picos de las
gráficas (ej. la letra más alta del cifrado seguramente es la 'e' o la 'a')."
    // Graficar (usando matplotlib)
    Crear_Grafica_Barras(
        datos_x = letras(ordenado_original),
        datos_y = conteos(ordenado_original),
        titulo = "Histograma Texto Original"
    )
    Crear_Grafica_Barras(
        datos_x = letras(ordenado_cifrado),
        datos_y = conteos(ordenado_cifrado),
        titulo = "Histograma Texto Cifrado"
    )
    Mostrar_Graficas_En_Pantalla()
```

Compilación y Ejecución

Para la primer parte del problema:

gcc P2_FabianGrajales.c -o P2_FabianGrajales

./P2_FabianGrajales

Para la segunda parte del problema:

- **Hay que usar la librería:**

pip3 install matplotlib

Compilación y ejecución:

python3 P2_FabianGrajales.py

Resultados:

Primera parte:

```
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema2$ gcc P2_FabianGrajales.c -o P2_Fabi
anGrajales
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema2$ ./P2_FabianGrajales
--- SISTEMA DE CIFRADO POR SUSTITUCIÓN ---
Clave de permutación generada y guardada en 'clave.txt'.
Alfabeto permutado: ñvbwukrdmolhgqxtfyjeicpnazs
Texto cifrado guardado en 'texto_cifrado.txt'.
Texto descifrado recuperado en 'texto_descifrado.txt'.
```

Segunda parte:

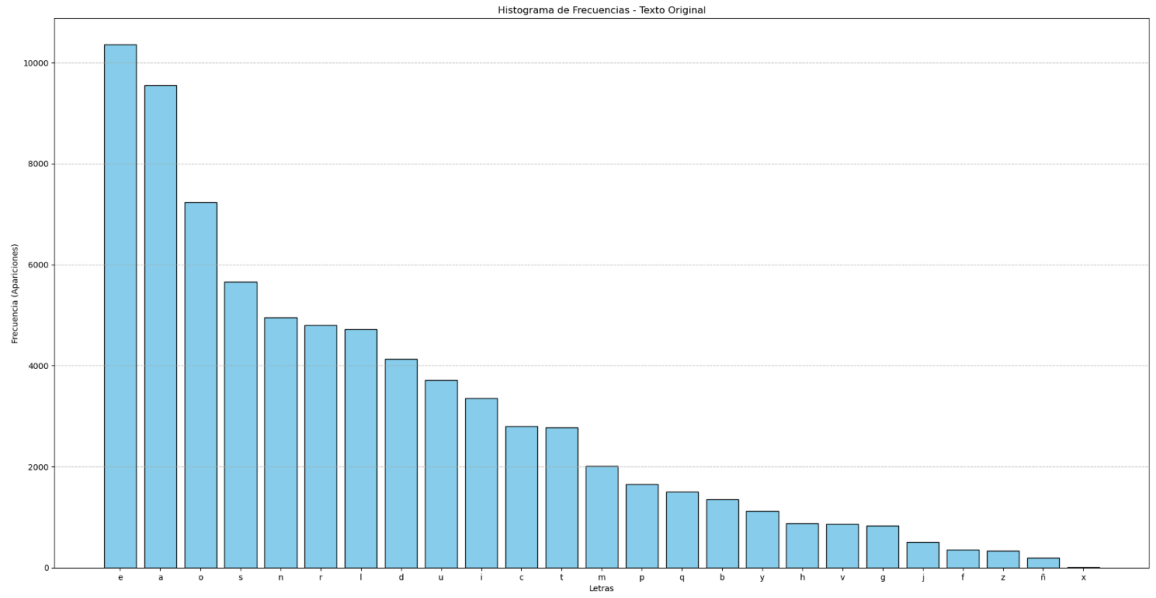
```
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema2$ python3 P2_FabianGrajales.py
--- ANÁLISIS DE FRECUENCIAS ---

Top 5 letras más frecuentes en el texto ORIGINAL:
  e: 10360 veces
  a: 9550 veces
  o: 7230 veces
  s: 5660 veces
  n: 4949 veces

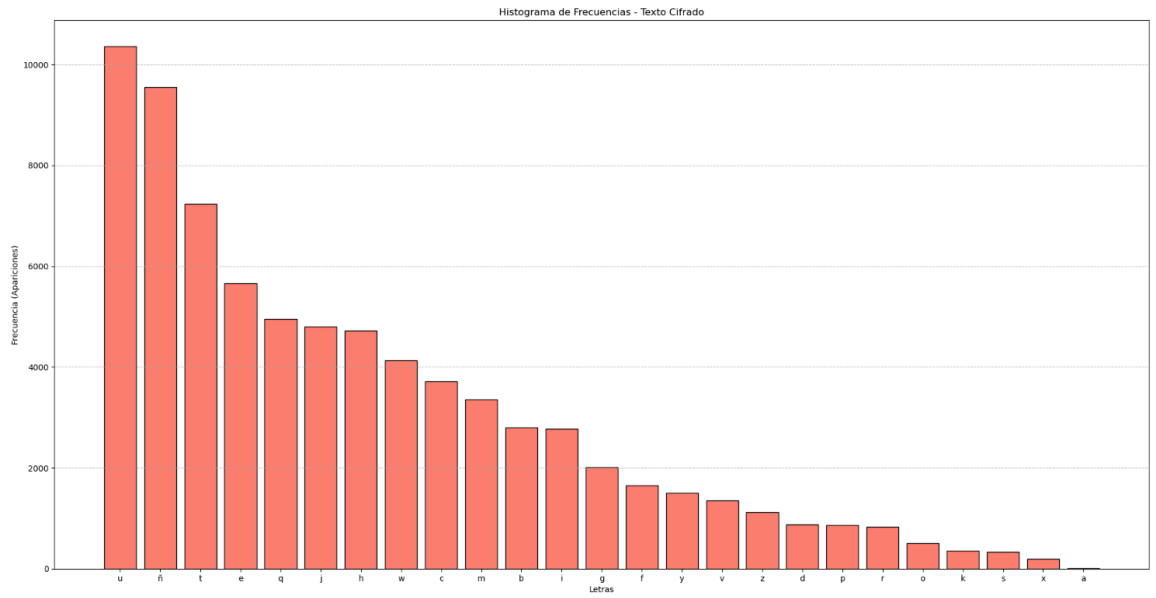
Top 5 letras más frecuentes en el texto CIFRADO:
  u: 10360 veces
  ñ: 9550 veces
  t: 7230 veces
  e: 5660 veces
  q: 4949 veces

¿Es posible descifrarlo? Sí.
Al comparar los histogramas (las letras con los picos más altos),
se puede deducir que la letra 'u' del texto cifrado
corresponde a la 'e' del original, y así sucesivamente.
```

Primera figura (Histograma del Texto Original): Representa el conteo de frecuencias de las letras del archivo original sin modificar. Esta gráfica muestra la distribución natural del idioma español:



Segunda figura (Histograma del Texto Cifrado): Representa las frecuencias de las letras en tu archivo ya encriptado.



Problema 3:

3. Implementar una lista doblemente ligada

- Utilice objetos como nodos (no estructuras de datos), es decir, el objeto Lista incluye objetos del tipo Nodo.
- Implemente constructores y destructores.
- Implemente un método para ordenar la lista (en base al atributo de su elección), agregar nodos, borrar nodos, insertar al inicio, final y una posición específica.
- Implemente la impresión de la lista (sobrecargar el operador ≪ tendrá valor extra).
- Sobrecarga el operador para unir dos listas. **Lenguaje: C++.**

Solución

Para abordar la implementación de la lista doblemente ligada, el diseño se estructura utilizando un enfoque puramente orientado a objetos donde el objeto Lista administra internamente objetos del tipo Nodo. La gestión dinámica de la memoria se controla estrictamente mediante la implementación de constructores y un destructor, asegurando que al destruir la lista se recorra cada elemento y se libere la memoria reservada evitando así fugas de memoria. En cuanto a la manipulación de la estructura, se definen métodos específicos para agregar nodos, borrar nodos, e insertar elementos tanto al inicio, al final, como en una posición específica, lo cual se logra reasignando los punteros siguiente y anterior de los nodos adyacentes para no romper la cadena. Para la funcionalidad de ordenar la lista en base a un atributo, la estrategia algorítmica más segura y eficiente consiste en recorrer la estructura intercambiando únicamente los valores internos de los nodos en lugar de modificar los punteros, previniendo así errores graves de segmentación. Finalmente, para lograr un uso más intuitivo del código, se implementa la impresión de la lista sobrecargando el operador << para enviar el recorrido de los datos directamente al flujo de salida estándar, y se sobrecarga el operador + con el objetivo de unir dos listas, generando una tercera estructura nueva que integra de forma secuencial los elementos de ambas listas originales sin alterar sus estados iniciales.

A manera de pseudocodigo:

```
CLASE Nodo:
    Atributos Privados:
        valor (Entero)
        siguiente (Puntero a Nodo)
        anterior (Puntero a Nodo)

    Constructor:
        Inicializar valor con el dato recibido
        siguiente = NULO
        anterior = NULO

    // Permitir que la clase Lista acceda a estos atributos privados
    AMISTAD con CLASE Lista


CLASE Lista:
    Atributos Privados:
        cabeza (Puntero a Nodo)
        cola (Puntero a Nodo)

    Constructor:
        cabeza = NULO
        cola = NULO

    Destructor:
        actual = cabeza
        MIENTRAS actual NO SEA NULO:
            siguiente_temp = actual.siguiente
            Liberar memoria de actual (delete)
            actual = siguiente_temp

    METODO insertar_al_inicio(dato):
        nuevo_nodo = CREAR Nodo(dato)
        SI lista_esta_vacia():
            cabeza = nuevo_nodo
            cola = nuevo_nodo
        SINO:
            nuevo_nodo.siguiente = cabeza
            cabeza.anterior = nuevo_nodo
            cabeza = nuevo_nodo

    METODO insertar_al_final(dato):
        nuevo_nodo = CREAR Nodo(dato)
        SI lista_esta_vacia():
            cabeza = nuevo_nodo
            cola = nuevo_nodo
        SINO:
            cola.siguiente = nuevo_nodo
            nuevo_nodo.anterior = cola
            cola = nuevo_nodo

    METODO insertar_en_posicion(dato, posicion):
```

```
    SI posicion == 0:
        insertar_al_inicio(dato)
        RETORNAR

    nuevo_nodo = CREAR Nodo(dato)
    actual = cabeza
    contador = 0

    MIENTRAS actual NO SEA NULO Y contador < posicion - 1:
        actual = actual.siguiente
        contador = contador + 1

    SI actual ES NULO O actual.siguiente ES NULO:
        insertar_al_final(dato)
    SINO:
        siguiente_temp = actual.siguiente

        nuevo_nodo.siguiente = siguiente_temp
        nuevo_nodo.anterior = actual

        actual.siguiente = nuevo_nodo
        siguiente_temp.anterior = nuevo_nodo

METODO borrar_nodo(dato_a_borrar):
    actual = cabeza
    MIENTRAS actual NO SEA NULO Y actual.valor != dato_a_borrar:
        actual = actual.siguiente

    SI actual ES NULO: // No se encontró
        RETORNAR

    SI actual == cabeza:
        cabeza = actual.siguiente
        SI cabeza NO ES NULO:
            cabeza.anterior = NULO
    SINO SI actual == cola:
        cola = actual.anterior
        SI cola NO ES NULO:
            cola.siguiente = NULO
    SINO:
        actual.anterior.siguiente = actual.siguiente
        actual.siguiente.anterior = actual.anterior

    Liberar memoria de actual

METODO ordenar():
    SI cabeza ES NULO O cabeza.siguiente ES NULO:
        RETORNAR // Ya está ordenada

    hubo_intercambio = VERDADERO
    MIENTRAS hubo_intercambio == VERDADERO:
        hubo_intercambio = FALSO
        actual = cabeza
```

```
            MIENTRAS actual.siguiente NO SEA NULO:
                SI actual.valor > actual.siguiente.valor:
                    // Intercambiar solo los datos, no los punteros
                    temp = actual.valor
                    actual.valor = actual.siguiente.valor
                    actual.siguiente.valor = temp
                    hubo_intercambio = VERDADERO
                actual = actual.siguiente

    SOBRECARGA OPERADOR + (otra_lista):
        nueva_lista = CREAR Lista()

        actual = this.cabeza
        MIENTRAS actual NO SEA NULO:
            nueva_lista.insertar_al_final(actual.valor)
            actual = actual.siguiente

        actual = otra_lista.cabeza
        MIENTRAS actual NO SEA NULO:
            nueva_lista.insertar_al_final(actual.valor)
            actual = actual.siguiente

        RETORNAR nueva_lista

// Fuera de la clase
SOBRECARGA OPERADOR << (flujo_salida, lista):
    actual = lista.cabeza
    MIENTRAS actual NO SEA NULO:
        flujo_salida << actual.valor << " <-> "
        actual = actual.siguiente
    flujo_salida << "NULO"
    RETORNAR flujo_salida
```

Compilación y ejecución:

g++ P3_FabianGrajales.cpp -o P3_FabianGrajales

./P3_FabianGrajales

Resultados:

```
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema3$ g++ P3_FabianGrajales.cpp -o P3_Fa
bianGrajales
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema3$ ./P3_FabianGrajales
--- PRUEBA DE LISTA DOBLEMENTE LIGADA ---
Lista 1 original: [ 10 <-> 20 <-> 50 <-> 30 <-> NULL ]
Lista 1 ordenada: [ 10 <-> 20 <-> 30 <-> 50 <-> NULL ]
Lista 1 tras borrar el 20: [ 10 <-> 30 <-> 50 <-> NULL ]

Lista 2: [ 100 <-> 200 <-> NULL ]

Lista 3 (L1 + L2): [ 10 <-> 30 <-> 50 <-> 100 <-> 200 <-> NULL ]
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema3$
```

Problema 4:
Programar un árbol en el que cada nodo puede tener n hijos (n > 2) y todos los nodos tienen la misma cantidad de hijos; al crear el árbol, se recibe como parámetro cuántos hijos tendría cada nodo. Debe programarse la funcionalidad para insertar nodos (utilice el criterio que le parezca adecuado, explique en el reporte) e imprimir todo el contenido del árbol. Utilice funciones recursivas.
Lenguaje: versión en C y versión en Python.

Solución:
Para abordar la creación de este árbol n-ario, la estructura principal consiste en un nodo que almacena su valor y un arreglo de tamaño exacto n para guardar los apuntadores a sus hijos. Al crear el árbol, el valor de n se establece como un parámetro global o de la clase, garantizando que todos los nodos respeten la misma capacidad ($n > 2$).
Criterio de Inserción: Se eligió un método de inserción recursiva por profundidad (Depth-First) de izquierda a derecha. Al intentar insertar un nuevo valor, la función recursiva evalúa los n espacios de los hijos directos del nodo actual; si encuentra un apuntador vacío (nulo), inserta el nuevo nodo ahí y termina. Si todos los hijos directos están ocupados, la función llama a la recursividad sobre el primer hijo, luego sobre el segundo, y así sucesivamente hasta encontrar el primer "hueco" disponible en las ramas inferiores. Este criterio cumple con la restricción de usar recursividad y garantiza que el árbol crezca de manera predecible, llenando las ramas izquierdas primero.
Impresión: Se diseñó un recorrido recursivo tipo Pre-orden. La función imprime primero el valor del nodo actual (añadiendo una sangría visual basada en la profundidad actual para facilitar la lectura) y luego itera sobre su arreglo de hijos, llamándose a sí misma recursivamente por cada hijo que no sea nulo.
A manera de pseudocodigo:

```
CLASE Nodo:
    Atributos:
        valor (Entero)
        hijos (Arreglo de tamaño 'n' con punteros a Nodos)

    Constructor(dato, n):
        valor = dato
        Para i desde 0 hasta n-1:
            hijos[i] = NULO
CLASE Arbol_N_Ario:
    Atributos:
        raiz (Puntero a Nodo)
        n (Entero, cantidad máxima de hijos por nodo)
    Constructor(capacidad):
        n = capacidad
        raiz = NULO
    // Función pública para insertar
    METODO insertar(dato):
        SI raiz ES NULO:
            raiz = CREAR Nodo(dato, n)
        SINO:
```

```
        insertar_recursivo(raiz, dato)
// Función privada recursiva para buscar lugar
METODO insertar_recursivo(nodo_actual, dato):
    SI nodo_actual ES NULO:
        RETORNAR FALSO
    // Intentar insertar en los hijos directos que estén vacíos
    Para i desde 0 hasta n-1:
        SI nodo_actual.hijos[i] ES NULO:
            nodo_actual.hijos[i] = CREAR Nodo(dato, n)
            RETORNAR VERDADERO // Inserción exitosa
          // Si todos los hijos directos están llenos, bajar al
              siguiente nivel
    Para i desde 0 hasta n-1:
        // Llamada recursiva
        SI insertar_recursivo(nodo_actual.hijos[i], dato) ES VERDADERO:
            RETORNAR VERDADERO
    RETORNAR FALSO // Por si acaso
// Función pública para imprimir
METODO imprimir_arbol():
    imprimir_recursivo(raiz, 0)
// Función privada recursiva para imprimir en Pre-orden
METODO imprimir_recursivo(nodo_actual, nivel):
    SI nodo_actual ES NULO:
        RETORNAR
    // Imprimir el nodo actual con sangría
    Imprimir (espacios según 'nivel') + nodo_actual.valor
    // Llamar recursivamente a todos sus hijos
    Para i desde 0 hasta n-1:
        SI nodo_actual.hijos[i] NO ES NULO:
            imprimir_recursivo(nodo_actual.hijos[i], nivel + 1)
```

Compilación y ejecución:

gcc P4_FabianGrajales.c -o P4_FabianGrajales

./P4_FabianGrajales

Resultados:

a) **Código en C**

```
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema4$ gcc P4_FabianGrajales.c -o P4_Fabi
anGrajales
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema4$ ./P4_FabianGrajales
--- ARBOL N-ARIO (n = 3) en C ---
Contenido del arbol:
-> 10
   -> 20
      -> 50
         -> 80
      -> 60
      -> 70
   -> 30
   -> 40
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema4$
```

b) Código en Python

```
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema4$ python3 P4_FabianGrajales.py
--- ÁRBOL N-ARIO (n = 3) en Python ---
Contenido del árbol:
-> 10
   -> 20
      -> 50
         -> 80
      -> 60
      -> 70
   -> 30
   -> 40
fabiangrajales@ASUS-TUF-F15:~/Documentos/Tareas - GitHub/Examen/Problema4$
```

A continuación se presenta un resumen de los problemas resueltos:

Problema 1: Simulación de Yahtzee

- Versión C++
    - Archivo fuente: P1_FabianGrajales.cpp
    - Comando de compilación: g++ P1_FabianGrajales.cpp -o P1_FabianGrajales
    - Comando de ejecución: ./P1_FabianGrajales
    - Archivos asociados: Genera P1_Resultados.txt
- Versión Python
    - Archivo fuente: P1_FabianGrajales_Python.py
    - Comando de compilación: (No requiere)
    - Comando de ejecución: python3 P1_FabianGrajales_Python.py
    - Archivos asociados: Genera P1_Resultados_Python.txt

Problema 2: Cifrado e Histogramas

- Versión C (Cifrado)
    - Archivo fuente: P2_FabianGrajales.c
    - Comando de compilación: gcc P2_FabianGrajales.c -o P2_FabianGrajales

    - Comando de ejecución: ./P2_FabianGrajales

    - Archivos:
        - Requiere texto_original.txt.
        - Genera: clave.txt, texto_cifrado.txt y texto_descifrado.txt

- Versión Python (Histograma)

    - Archivo fuente: P2_FabianGrajales.py

    - Comando de compilación: (No requiere)

    - Comando de ejecución: python3 P2_FabianGrajales.py

    - Archivos asociados:
        - Requiere los textos generados en C.
        - Genera las gráficas de Matplotlib.

Problema 3: Lista Doblemente Ligada

- Versión C++

    - Archivo fuente: P3_FabianGrajales.cpp

    - Comando de compilación: g++ P3_FabianGrajales.cpp -o P3_FabianGrajales

    - Comando de ejecución: ./P3_FabianGrajales

    - Archivos asociados: (Salida en consola)

Problema 4: Árbol N-ario

- Versión C

    - Archivo fuente: P4_FabianGrajales.c

    - Comando de compilación: gcc P4_FabianGrajales.c -o P4_FabianGrajales

    - Comando de ejecución: ./P4_FabianGrajales

    - Archivos asociados: (Salida en consola)

- Versión Python
    - Archivo fuente: P4_FabianGrajales.py
    - Comando de compilación: (No requiere)
    - Comando de ejecución: python3 P4_FabianGrajales.py
    - Archivos asociados: (Salida en consola)